

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	SARU HASINI.S	Reg.No.:	192521418
Department:	BTECH.IT	Date:	31/07/2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I S A R U H A S I N I . S (1 9 2 5 2 1 4 1 8) __certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SARU HASINI.S

MARKS ALLOCATION

	Problem Understanding(20%)	Method Selection(20%)	Solution Process(25%)	Accuracy of Calculation(20%)	Interpretation of Results(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. 2's Complement Addition and Subtraction (+45 and -23)

Binary Representation (8-bit)

+45 = 00101101

+23 = 00010111

-23 = 2's complement of 23

00010111

1's complement = 11101000

+1 = 11101001

Addition (+45 + (-23))

00101101
+ 11101001

00010110

Result: 00010110 = 22

Subtraction (+45 - (-23))

= 45 + 23

00101101
+ 00010111

01000100

Result: 01000100 = 68

Overflow

No overflow occurs because the result is within -128 to +127.

Overflow is detected when adding two numbers with the same sign gives a result with a different sign.

2. Carry Look-Ahead Adder (CLA)

Generate and Propagate

Generate: $G = A \times B$

Propagate: $P = A \oplus B$

Carry Equations

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

Comparison

Ripple Carry Adder

Carry Look-Ahead Adder

Carry moves one bit at a time

Carries calculated together

Slower

Faster

More delay

Less delay

Conclusion

CLA improves processor speed by reducing carry propagation delay.

3. Booth's Algorithm ($-6 \times +5$)

Binary

$$-6 = 1010$$

$$+5 = 0101$$

Initial Values

$$A = 0000$$

$$Q = 0101$$

$$M = 1010$$

$$Q_{-1} = 0$$

Steps

Check Q_0 and Q_{-1} .

$$10 \rightarrow A = A - M$$

$$01 \rightarrow A = A + M$$

00 or 11 $\rightarrow$ No operation.

Perform arithmetic right shift.

Repeat for 4 cycles.

Result

$$-6 \times 5 = -30$$

Advantages

Handles signed numbers directly.

Reduces addition/subtraction operations.

Faster than ordinary multiplication.

4. Restoring and Non-Restoring Division ($13 \div 3$)

Binary

$$\text{Dividend} = 1101$$

$$\text{Divisor} = 0011$$

Restoring Division

Shift left.

Subtract divisor.

If remainder is negative, restore and write quotient bit = 0.

Otherwise quotient bit = 1.

Result

$$\text{Quotient} = 0100 \text{ (4)}$$

$$\text{Remainder} = 0001 \text{ (1)}$$

Non-Restoring Division

Shift left.

If previous remainder positive $\rightarrow$ subtract divisor.

If negative $\rightarrow$ add divisor.

No restoring after every subtraction.

Result

$$\text{Quotient} = 0100 \text{ (4)}$$

$$\text{Remainder} = 0001 \text{ (1)}$$

Comparison

Restoring

Non-Restoring

More operations

Fewer operations

Slower

Faster

Higher complexity

More efficient

5. IEEE 754 Floating Point (-13.25)

Decimal to Binary

$$13.25 = 1101.01_2$$

Normalized form:

$$1.10101 \times 2^3$$

Single Precision (32-bit)

Sign = 1

$$\text{Exponent} = 3 + 127 = 130 = 10000010$$

Mantissa = 101010000000000000000000

Double Precision (64-bit)

Sign = 1

$$\text{Exponent} = 3 + 1023 = 1026 = 10000000010$$

Mantissa = 10101000

Floating-Point Addition Steps

Compare exponents.

Align mantissas.

Add/Subtract mantissas.

Normalize the result.

Round if required.

Store sign, exponent, and mantissa.

Issues

Precision loss

Rounding errors

Normalization required

Overflow/Underflow possible